Architectural Patterns for Integrating LLMs into User-Facing Applications

Lessons from a language-learning platform

MIRCEA LUNGU, IT University of Copenhagen, Denmark

Large Language Models are increasingly being integrated as components into existing user-facing applications, alongside the more visible wave of LLM-native products such as chatbots and agents. The engineering concerns of the two cases differ: when an LLM is a component behind an existing feature rather than the product itself, designers must reconcile its per-token cost, multi-second latency, non-determinism, rapidly shifting provider landscape, and general-purpose capability with the expectations of users who already rely on the system. We present a catalogue of recurring architectural patterns that address these concerns, grounded in over a year of LLM integration work on Zeeguu, an open-source language-learning platform with several hundred monthly active users. The catalogue spans five concerns — cost optimization, latency and availability, lifecycle management, data management, and quality assurance — and is described using the standard pattern format. The patterns are presented as a starting point for community refinement and extension, not as a closed taxonomy.

Additional Key Words and Phrases: large language models, software architecture, design patterns, LLM integration, cost, latency, quality assurance

1 Introduction

Large Language Models are increasingly being integrated into existing user-facing interactive applications, not as standalone chatbots, but as components working behind the scenes to improve the user experience. While there is growing literature on building LLM-native products (chatbots, agents, RAG systems) and on using LLMs for code generation, there is surprisingly little guidance on the **software engineering challenges of integrating LLMs as components into existing interactive applications** where real users expect fast, reliable, and trustworthy responses.

LLMs have a unique combination of properties that create novel architectural forces:

- they are expensive (per-token pricing),
- slow (high latency),
- non-deterministic (same input can yield different outputs, although this can be to a certain degree controlled with the temperature setting),
- rapidly evolving (new models released every few months and old ones regularly deprecated),
- imprecise (they make mistakes), and
- general-purpose (they can attempt almost any task described as text).

These properties demand specific engineering strategies.

Over the past year, we have been integrating LLMs into Zeeguu, an open-source platform for personalized language learning that helps users learn foreign languages by reading authentic online content. Through this work, we have

Author's Contact Information: Mircea Lungu, IT University of Copenhagen, Copenhagen, Denmark, mircea.lungu@gmail.com.

identified a set of recurring architectural patterns for LLM integration. We believe these patterns are general and applicable beyond our specific domain.

2 Case Study: Zeeguu

Zeeguu is an open-source language learning platform built around the idea that learners benefit most from engaging with authentic content in their target language. Rather than relying on artificial textbook exercises, Zeeguu helps users find real articles (news, blog posts, and other web content) tailored to both 1) their *level* and 2) their *interests*.

The platform recommends articles in the learner's target language based on their proficiency and reading preferences, making it **easy to find material that is both engaging and appropriately challenging**. If a text is personally compelling but too difficult, Zeeguu simplifies it to the learner's level using LLMs.

When users encounter unfamiliar words or phrases while reading, they can get translations on the fly powered by (contextual) Google Translate, so the reading experience remains fluid and uninterrupted. One alternative is provided by a state-of-the-art LLM, offering a more contextually nuanced option.

Every translation a user requests is logged by the system, which over time builds a **detailed model of the learner's vocabulary knowledge**, tracking which words they know, which ones they struggle with, and how well they've retained previously encountered vocabulary.

Based on this evolving learner model, Zeeguu **generates interactive vocabulary exercises and audio lessons** that focus on the words that matter most for each individual learner, rather than following a generic curriculum. The exercises use the context in which the word was originally encountered, based on the assumption that if the original text was compelling to the learner, examples drawn from it will be too.

In essence, Zeeguu unifies reading, translation, learner modeling, and practice into a coherent pipeline, with the learner's own reading interests as the primary driver.

Zeeguu currently serves over 300 monthly active users, with peaks exceeding 400 during the academic year[1].

3 Cost Optimization

We use *cost* as shorthand for any expensive, metered resource — most visibly the per-token API bill, but also latency, compute, and energy. The unifying force is that LLM calls carry a large, often *fixed* overhead (the instructional prompt, the network round-trip), so invoking them naïvely wastes that setup on a tiny payload.

These patterns are really about *economy*: **amortizing a fixed overhead** across many uses (*Prompt Amortization*), and **paying the expensive resource only in proportion to the value returned** (*Escalate to the LLM*). The same frugality transfers to non-monetary resources — which is why Prompt Amortization cuts latency as well as dollars.

3.1 Prompt Amortization

3.1.1 Example. Several Zeeguu jobs share the same shape (a large instructional prompt wrapped around a tiny variable input, see figure) and run offline over many items. Instead of paying that preamble once per item, related items are packed into a single call:

- **Meaning classification** sends ~15 word-meanings per call, sharing one frequency/CEFR-type taxonomy prompt across the whole batch.
- **Example-sentence validation** checks ~20 generated examples per call.

[1] Monthly active users are defined as users with any learning activity (exercises, reading, browsing, audio lessons, or translations) in a given month. Live statistics are available at: https://api.zeeguu.org/stats/monthly_active_users

```
COMBINED_VALIDATION_PROMPT = """You are validating and classifying a language learning translation.

A learner highlighted "{word}" in this {source_lang} sentence:
"{context}"

The translation is in {target_lang}: "{translation}"

CRITICAL: The translation "{translation}" is in {target_lang}, NOT English!
If {target_lang} is Dutch, German, etc., interpret the translation IN THAT LANGUAGE.
Example: "offer" in Dutch means "victim/sacrifice" - this is VALID for "víctima" (Spanish).

STEP 1 - VALIDATION:
1. Is "{word}" a meaningful learning unit? (Not an arbitrary fragment like "bruger det" = "are using it")
2. Is "{translation}" a correct translation for the WORD itself (not the surrounding phrase)?
3. If the word is part of an idiom, decide: should the learner study the single word or the full idiom?

IMPORTANT: Only mark as FIX if the translation is WRONG. Do NOT "improve" correct translations.
- Keep translations SHORT and SIMPLE - learners must type them in exercises
- NO parenthetical explanations like "by (doing something)" - just "by"
- NO alternatives with "/" like "ea/in the capacity of" - pick ONE: "as"
- NO articles unless essential: "eyes" not "the eyes"

STEP 2 - CLASSIFICATION (for the valid/corrected word):
Frequency:
- unique: only meaning of the word
- common: primary or frequently used meaning
- uncommon: infrequently used meaning
- rare: specialized, archaic, or context-specific

CEFR Level (what level learner should know this word):
- A1: basic everyday words (hello, cat, eat)
- A2: common everyday vocabulary (weather, shopping)
- B1: intermediate topics (opinions, work, travel)
- B2: abstract concepts, nuanced vocabulary
- C1: advanced, sophisticated vocabulary
- C2: rare, literary, highly specialized

Phrase type:
- single_word: individual word
- collocation: natural word combination ("strong coffee", "take place")
- idiom: non-literal meaning ("break the ice", "piece of cake")
- expression: common phrase/greeting ("how are you")
- arbitrary_multi_word: random fragment, NOT worth studying ("doctor for", "the cat on", "bruger det")

Reply in this EXACT format (one line):
VALID|frequency|cefr|phrase_type|explanation|literal_meaning
or
FIX|corrected_word|corrected_translation|frequency|cefr|phrase_type|reason|explanation|literal_meaning

Fields:
- cefr: A1, A2, B1, B2, C1, or C2
- explanation: OPTIONAL usage notes, register (formal/informal). Leave empty if not needed.
- literal_meaning: ONLY for idioms - SHORT word-by-word translation (e.g., "kick into touch").
  Leave EMPTY for single words, collocations, and non-idioms. NOT for explanations!

Examples:
- Simple word: VALID|common|A1|single_word||
- Word with usage note: VALID|common|B1|single_word|formal register|
- Idiom: VALID|common|B2|idiom|kick into touch
- Wrong translation: FIX|øjnene|the eyes|common|A2|single_word|literal meaning is 'the eyes'||
- Idiom fix: FIX|se virkeligheden i øjnene|face reality|common|B2|idiom|idiom meaning 'face reality'||see reality in the eyes
- Arbitrary fragment: FIX|bruger|use|common|A2|single_word|'bruger den' is arbitrary fragment||
"""
```

Fig. 1. The `COMBINED_VALIDATION_PROMPT` template: ~250 lines of validation rules, frequency/CEFR/phrase-type taxonomies, output format, and examples, wrapped around just three variables (`{word}`, `{translation}`, `{context}`). Sent one pair at a time, the entire preamble is re-paid on every call. This fixed overhead is the cost the pattern amortizes.

- **Article simplification** produces every CEFR level simpler than the original in one call, one section per level, turning four or five requests into one (~75% fewer calls for a typical article).

3.1.2 Forces.

- **Preamble overhead.** A large, fixed instructional prompt wrapped around a tiny variable input; sent one item at a time, that preamble is re-paid on every call, in tokens, cost, and latency alike. *(pushes toward bigger batches)*
- **Quality ceiling.** Accuracy and consistency degrade as more items share one call; past ~15–20 small items some models start dropping or muddling entries. *(pushes toward smaller batches)*
- **Context ceiling.** Input *and* output must fit the window; for fan-out the output side binds first, since each result is full-length.
- **Interactive latency.** Fan-in requires waiting to accumulate enough items to fill a batch: fine offline, but unacceptable when a user is blocked on a single result.

3.1.3 Solution. Batch multiple items into a single request, amortizing the expensive prompt across all of them. This takes two forms:

- ***Fan-in* batching** packs many independent inputs into one prompt, spreading a large instructional preamble across the whole batch.
- ***Fan-out* batching** produces many outputs from a single input, emitting one section per output and collapsing several requests into one.

Both combine naturally with pre-computation: because results are computed offline, there is the luxury of batching.

3.1.4 Code.

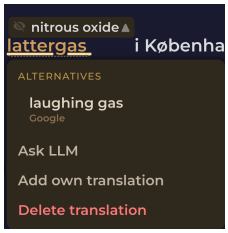


Fig. 2. In Zeeguu, the inline Google translation is the primary path; when the user wants a better rendering they escalate to an LLM on demand via the "Ask LLM" option.

- `COMBINED_VALIDATION_PROMPT`. The per-pair validation prompt: the "substantial instructions" (what counts as a valid translation, edge cases, output format). It runs once *per word*: the large preamble this pattern exists to amortize.
- `create_batch_meaning_frequency_and_type_prompt`. Fan-in batching: ~15 meanings classified in one call.
- `validate_examples_batch`. Fan-in batching: generated examples validated ~20 at a time (`BATCH_SIZE`).
- `get_adaptive_simplification_prompt`. Fan-out batching: one call produces simplified versions for *all* CEFR levels simpler than the original.

3.1.5 *Notes.*

- Which ceiling binds first? For small items it is the *quality* ceiling, not the token one: classification and example validation run at **15–20 items per call**, far below what the window allows, because beyond that the model starts dropping or muddling entries. For fan-out simplification it flips: the *token* ceiling binds on the output side, since each variant is a full article.
- Some LLMs provide prompt caching - e.g. Deepseek. Even so, if the cost is amortized with prompt caching, the time saving of amortization can still be a valuable reason for doing it

3.1.6 *Known Uses.*

- **Batch prompting [3]** (Cheng, Kasai & Yu, EMNLP 2023) packs multiple independent samples under one shared instructional prompt in a single call, cutting token and time cost roughly inverse-linearly with batch size.
- **Vertical batching** maps to structured-output calls that emit several keyed results at once (and to the OpenAI/Anthropic multi-output `n` parameter).
- *Distinguish from provider batch APIs.* The OpenAI Batch API and Anthropic Message Batches give ~50% off large asynchronous jobs, but each request still carries and pays for its own full prompt — they amortize scheduling and rate-limit overhead, *not* the in-prompt instructional overhead this pattern targets.

3.2 Escalate to the LLM

3.2.1 Example. Google Translate serves as the primary translation engine. When a user indicates the translation is inadequate, the system escalates to an LLM for a more nuanced, context-aware translation. This keeps costs low and speed high in the common case while providing higher, LLM-quality results when needed.

3.2.2 Forces. Specialized tools (translation APIs, NLP pipelines, classical classifiers) are faster, cheaper, and more deterministic than LLMs for well-defined tasks, but they sometimes fail or produce insufficiently satisfactory results.

3.2.3 Solution. Use the specialized tool as the primary path and escalate to the LLM only when the primary fails or the user signals dissatisfaction.

3.2.4 Applicability. This pattern applies broadly: topic classification, named entity recognition, or any NLP task where a cheaper tool handles the common case and the LLM handles the long tail.

3.2.5 Notes.

- *Escalation, not fallback.* Unlike a reliability fallback (where the secondary is an equal-or-lesser backup invoked when the primary fails; see *Fail-Fast Provider Chain*), here the secondary is **more capable and more expensive**, invoked when the primary is not good enough. The movement is *up* in quality and cost, not *down* into degraded mode. That is why we name it escalation.
- *Relationship to the model cascade.* This is the human-/failure-triggered cousin of the **model cascade** in ML serving, where a cheap model runs first and a confidence threshold routes hard inputs to a larger model. The shared shape is *cheap tier first, expensive tier on demand*; the difference is the trigger. A cascade escalates automatically on the model's own low confidence, whereas this pattern escalates on external signals: the primary tool erroring, or the user explicitly declaring the result inadequate. A confidence-based cascade is thus one possible escalation policy; user dissatisfaction is another, and the two can be combined.

3.2.6 Known Uses.

- **FrugalGPT [2]** (Chen, Zaharia & Zou, 2023) queries cheaper models first and escalates to more capable, expensive ones only when a scorer rejects the cheap answer.
- **RouteLLM [9]** (Ong et al., 2024) trains a router that sends easy queries to a weak/cheap model and escalates only hard ones to the strong model; shipped as an open-source framework.
- These are the *model-cascade* cousins of the pattern — automatic, confidence-triggered — whereas Zeeguu's trigger is a failure of the cheaper *non-LLM* tool or explicit user dissatisfaction.

3.3 Self-Hosted Slow-Path Inference

3.3.1 Example (planned). Overnight, a local LLM on a server that belongs to the system creator (e.g. a Mac Studio) drains a queue of slow, batchable jobs that today run against paid APIs (translation validation, example-sentence checking, CEFR pre-classification).

- A worker on the Mac Studio polls the Zeeguu server over outbound HTTPS, runs each job on a local model (e.g. via Ollama), and posts the result back.
- Anything not processed by a morning deadline falls back to the cloud API.

3.3.2 Forces. API usage is the dominant variable cost of an LLM integration, yet a large share of the work is latency-insensitive: pre-computed, batched, offline (see *Pre-Computing Likely-Needed Results*, *Prompt Amortization*).

Capable open models now run on prosumer hardware (for example a Mac Studio with large unified memory) that is already owned and sits idle at night. For some tasks (e.g. example sentence generation) one does not need to use the latest online models.

The obstacle is connectivity: such a machine usually sits behind NAT with no public IP, and opening inbound ports adds an attack surface its owner does not want.

3.3.3 Solution. Run the latency-insensitive tasks on a local model on the owned hardware, and connect it outbound-only. The server enqueues jobs; a worker on the home machine polls that queue over HTTPS, runs each job on a local runtime, and posts the result back. The home machine exposes nothing: no public IP, no inbound ports, no listening service, only outbound calls to the server that already exists.

3.3.4 Consequences. Trades API cost for sunk hardware and electricity, on the slow path only; real-time paths still use the cloud. Local model quality and throughput are lower, so the cloud API stays as a deadline-bound fallback (composes with *Fail-Fast Provider Chain* and *Escalate to the LLM*): if the home worker has not drained the queue by the time results are needed, the cloud takes over. Availability is best-effort, since the machine may be asleep or off, so only work that can wait is eligible. Results carry a different model identity and should be recorded as such (composes with *LLM Output Provenance*).

3.3.5 Notes.

- *Outbound-only is the key.* A pull-based worker exposes nothing and is the smallest attack surface. If the server must instead call the local model synchronously, a mesh VPN such as Tailscale (or a Cloudflare Tunnel) gives the server a route to the machine without port-forwarding or a public IP.
- *Owned versus volunteered.* Here the hardware belongs to the operator. A more ambitious variant accepts compute volunteered by third parties, which adds a trust dimension: outputs from untrusted workers must be validated before use (composes with *LLM Content Validation Tracking*) and may need cross-checking across workers.

3.3.6 Known Uses.

- **Ollama / llama.cpp on Apple Silicon** are routinely used to run extraction/summarization batch jobs overnight on an owned Mac with no per-token cost and no rate limits — the slow-path use directly.
- **Apple Intelligence + Private Cloud Compute** ships a local-first / cloud-fallback split, escalating to a larger cloud model only when needed — the same local+cloud division of labour, though *inverted*: Apple uses local as the fast path, this pattern uses it as the slow/cheap path.
- **GitHub Actions self-hosted runners** are the outbound-only pull-worker mechanism: the runner long-polls over HTTPS so "only an outbound connection… is required," exposing no inbound ports — exactly the NAT-traversal design proposed here.
- *Novel composition.* Each mechanism is well-attested independently; combining local slow-path + cloud deadline-fallback + outbound-only worker into one LLM system is this candidate's contribution.

3.4 Per-User Consumption Budget

3.4.1 Example. Several of Zeeguu's LLM actions are triggered *directly by a user's click* and charge a shared provider account the instant the user acts: on-demand "Ask LLM" translation, on-demand article simplification, and audio-lesson generation (LLM script + text-to-speech). Because the spend is attached to individual user behaviour, a single enthusiastic user — or a buggy client, or a script — can run up the bill or exhaust shared rate limits for everyone.

Zeeguu already contains a crude instance of the defense. Audio-lesson generation allows **only one active generation per user at a time**: `AudioLessonGenerationProgress.create_for_user` deletes any existing record and `find_active_for_user` gates new requests. That is a per-user *concurrency* budget of exactly one — the cheapest possible bound — chosen precisely because the audio pipeline is among the most expensive actions in the system.

The general pattern extends this idea to any LLM-backed resource: cap each user's consumption over a time window, denominated in whatever proxy is cheap to measure and *good enough* — number of on-demand translations per day, characters simplified per week, generations per hour, or, at the precise end, actual token cost.

3.4.2 Forces.

- On-demand LLM actions cost real money and latency per invocation, and — unlike a pre-computed or cached feature — they are driven by user behaviour, so from the system's side consumption is unbounded.
- Consumption is wildly uneven across users. A small number of heavy users, an abusive script, or a runaway client can dominate cost and exhaust the shared provider's rate limits, degrading service for everyone.
- The bound must be per *user* (the entity the spend is attached to), not global, or one user's overuse silently taxes the rest.
- Exact cost accounting is precise but comparatively expensive to build (capture token usage, maintain a current price table). Often a coarse proxy — a count, a concurrency cap, a time budget — is far cheaper and adequately bounds the failure you actually care about.

3.4.3 Solution. Give each user a budget on an LLM-backed resource and refuse or degrade once it is exhausted. Denominate the budget in the **coarsest proxy that adequately tracks the risk**:

- **concurrency** — at most N in flight per user (the audio-lesson case, $N = 1$);
- **count over a window** — translations per day, simplifications per week;
- **volume** — characters simplified, tokens consumed;
- **actual cost** — the precise variant (see below).

When a budget is exhausted, **degrade rather than fail** where possible: fall back to the cheaper non-LLM path (serve the Google Translate result and simply stop escalating to the LLM — see *Escalate to the LLM*), queue the request, or show a friendly "try again later."

Precise variant — Per-User Cost Attribution. At the accurate end, meter every LLM call as `(user, feature, model, provider, input_tokens, output_tokens, timestamp)`, store **tokens** (provider-neutral) and derive **cost** through a central price table, and aggregate per user. This is the most accurate budget denomination and doubles as observability — it answers *which users and which features drive the bill*. It also composes with *Centralized Model Selection* (price table keyed by the same model constants) and *LLM Output Provenance* (the per-artifact provenance record is the natural place to also stamp token counts: provenance says *how was it made*, this adds *what did it cost, and to whom*). Prefer a coarse proxy unless you genuinely need this precision.

3.4.4 Notes.

- Choose the coarsest proxy that prevents the failure you care about. If the risk is "a script hammers *simplify*," a per-hour count stops it; you do not need per-token accounting.
- Attribute to the provider that **actually** served the call — a *Fail-Fast Provider Chain* fallback changes who you pay and at what rate.

- The graceful-degradation clause is what keeps this user-friendly rather than punitive: a language learner who exhausts their LLM-translation budget still gets Google Translate, not an error.

3.4.5 Relationship to LLM Gateways. Gateways provide per-key / per-user rate limiting and spend caps out of the box (LiteLLM virtual-key budgets and RPM/TPM limits, Portkey, Cloudflare AI Gateway), so both the coarse and the cost-based variants can be *partly* delegated — but only if the application tags each call with a user id (and, for per-feature budgets, with the feature). As with metering generally (see *What Makes These Patterns LLM-Specific? → Relationship to LLM Gateways*), the gateway supplies the **enforcement primitive** (count, throttle, cap); the pattern owns the **policy** — which resource, which window, and crucially *what the user gets when the budget is exhausted* (degrade to Google Translate vs. hard refuse), which is a product decision no gateway can make.

3.4.6 Status. A crude instance (single active audio generation per user) already ships. The general per-user budget across on-demand actions, and the cost-accounting variant, are not yet implemented — included as a candidate to invite discussion on whether this is one pattern with several denominations (concurrency / count / volume / cost) or several distinct patterns.

3.4.7 Known Uses.

- **LiteLLM Proxy** supports per-key `max_budget` (with auto-reset), per-user budgets across a user's keys, and rpm/tpm/parallel-request caps.
- **Portkey** attaches a USD budget and rate limits to an individual key, expiring it when the budget is exhausted.
- **Helicone** rate-limits per user via a policy header denominated in request count or cost (e.g. "$5.00/hour/user").
- *Note.* All three are gateway primitives: they enforce the *mechanism* but not the domain *policy* (which resource, and what the user gets when exhausted) — consistent with this pattern's gateway discussion.

4 Latency and Availability

4.1 Pre-Computing Likely-Needed Results

4.1.1 Example. When a learner reads a text and asks for a translation, the most important thing is to return a good-enough translation **fast**. The system can afford an imprecise translation while the reader is quickly making sense of a text, but it cannot afford to have learners repeatedly *practice* an imprecise one.

The vocabulary exercises are built from the words a learner has looked up, so those pairs (from Google or Azure Translate) must be verified before they enter an exercise. Verifying with an LLM is too slow to run at the moment a learner opens a session. So a regular cron job looks ahead: it identifies the words a learner is due to study next and pre-computes the LLM verification, so that when the session starts the words are already vetted and ready, with no LLM call on the critical path.

An even costlier instance: audio lessons. Generating a personalized audio lesson is more expensive again: an LLM writes the lesson script, then text-to-speech synthesizes the audio, several seconds of work no learner should wait through. A nightly job pre-computes the next lessons for recently active learners (prioritized by how recently they practiced), on the assumption that someone who has been studying will be back for the next one. When they return, the lesson is already waiting.

4.1.2 Forces. LLMs can provide valuable data for users, but they are slow and expensive, making their invocation impractical when the user needs an answer in real-time. (Real-time users expect answers in 200ms, while depending

on the prompt and the deployment configuration, an LLM-based system can take multiple seconds to produce an answer).

4.1.3 Solution. Anticipate likely user needs and pre-compute LLM results offline (e.g., via cron jobs), so results are available instantly when needed. The system designer should model user behavior in order to predict their LLM needs.

4.1.4 Known Uses.

- **ColBERT [6]** (Khattab & Zaharia, SIGIR 2020) precomputes contextualized passage embeddings for the whole corpus offline, so query time runs only cheap late-interaction matching — the canonical "precompute the expensive representation ahead of time."
- Provider **batch / offline-inference** pipelines are the enabling infrastructure for computing LLM outputs in bulk off the hot path.
- *Honest gap.* We did not find a well-documented production system that precomputes *user-facing* LLM answers on a schedule the way Zeeguu does; the closest analogues are the materialized-view / prefetching principle and the offline-index precomputation above.

4.2 Hot-Path Result Caching

4.2.1 Example. Multi-word expression (MWE) detection finds the phrases in an article that a learner might want translated as a unit (for example *kick the bucket*). It uses an LLM, gated by a cheap Stanza pass (see *Hybrid Classical+LLM Pipeline*), and the LLM call is the expensive part, so the analysis is worth caching: the detector keeps a 500-entry in-memory cache, so when multiple users read the same article, phrase analyses computed for the first reader are served instantly to the rest. Hit rates are highest for popular articles that many users read in the same window.

4.2.2 Forces. Pre-computation handles predictable needs, but some LLM queries are repeated unpredictably within short time windows (e.g., multiple users encountering the same phrase, or a single user re-requesting the same analysis). These don't justify persistent storage but do benefit from short-term caching.

4.2.3 Solution. Maintain an in-memory LRU cache for recent LLM results. Cache keys include the relevant input parameters; cache entries expire after a short TTL or when capacity is reached.

4.2.4 Tradeoff. Memory overhead and cache invalidation complexity. Best suited for queries where staleness is acceptable and input space has natural clustering (many users reading same content).

4.2.5 Tradeoff.

- A variant of this caches the results in the DB not in-memory. We also have this in Zeeguu.

4.2.6 Known Uses.

- **GPTCache [1]** (Bang, NLP-OSS @ EMNLP 2023) is a dedicated LLM cache supporting both exact-match and semantic (embedding-similarity) lookup.
- **Helicone** caches on a hash of the request, stored at the edge with a configurable TTL.
- **Portkey** ships both "simple" (exact-match) and "semantic" caches; **LangChain** provides in-memory and SQLite caches.

4.3 Multiplexed Dispatch

4.3.1 Example. Real-time translations are dispatched to multiple translation providers in parallel. The first response is used, and the rest are saved for when the user asks for alternatives.

4.3.2 Forces. Multiple LLM providers offer similar capabilities but with varying latency. When speed matters for user experience, relying on a single provider creates a bottleneck.

4.3.3 Solution. Dispatch the same request to multiple providers simultaneously and use the first response that arrives. Track the top two fastest providers, and always dispatch to these.

An alternative to this is **live retrieval**: when the user encounters a translation that they are not sure of, they ask for alternatives, the UI presents the results that are cached, but also asks for alternatives and displays a UI interface that highlights the fact that some of the results are still streaming in.

4.3.4 Tradeoff. Increased cost (paying for redundant calls) in exchange for reduced latency.

4.3.5 Note. In itself this is not LLM-specific. It is the *hedged requests* pattern from distributed systems (Dean and Barroso, *The Tail at Scale*, CACM 2013), also known as request racing or tied requests. Two things give it an LLM flavour here. 1. First, the hedge is across independent, competing vendors (Anthropic, DeepSeek, Google Translate) rather than replicas of a single service. 2. Second, each redundant call costs real money per token, so the losing responses are not thrown away: they are kept and surfaced to future readers of the same text as alternative translations, turning the redundant work into a feature.

4.3.6 Known Uses.

- **Hedged requests [4]** (Dean & Barroso, "The Tail at Scale," CACM 2013) are the classical ancestor: send a duplicate to another replica after a latency threshold, take the first, cancel the rest — cutting BigTable p99 from 1800ms to 74ms at ~2% extra load.
- General infrastructure ships it: **gRPC `hedgingPolicy`** and **Polly** (.NET) race concurrent copies and take the first response.
- *Honest gap / opportunity.* We could not find a major LLM gateway that ships true provider *racing* as a first-class feature — most do sequential fallback (see *Fail-Fast Provider Chain*) or learned single-model routing. Zeeguu's parallel dispatch across translation providers appears to be an under-adopted transfer of the hedging idea to LLMs.

4.4 Fail-Fast Provider Chain

4.4.1 Example. (Zeeguu): The app uses a unified LLM service proxy which tries Anthropic first; on any error (timeout, rate limit, API error), it immediately switches to DeepSeek without retry. This keeps worst-case latency bounded while maintaining availability.

4.4.2 Forces. LLM providers experience outages, rate limits, and latency spikes. Retry logic increases latency and may still fail. Different providers offer similar capabilities at different price/performance points.

4.4.3 Solution. Configure a chain of LLM providers with no retries. On any failure, immediately fall back to the next provider in the chain. Prioritize speed over exhausting retry budgets.

4.4.4 Applicability. The pattern earns its keep most on **real-time paths where a user is waiting**. There the latency budget is tight, so spending it on a retry against a failing provider is the wrong move: failing over immediately keeps the worst case bounded. For offline or batched work, where nobody is waiting, retries and backoff are more affordable, and it can be worth retrying the cheaper provider before moving on.

4.4.5 Note. This differs from *Escalate to the LLM* in that all components in the chain are LLMs offering equivalent capabilities: this is a *fallback* for reliability, not the *escalation* to a more capable (and more expensive) tier.

4.4.6 Known Uses.

- **Portkey** fallback mode walks a prioritized provider list, switching on the first non-2xx response (retries are a separate opt-in).
- **Cloudflare AI Gateway** tries providers sequentially, falling to the next on error or timeout, and reports which provider served via a response header.
- **LiteLLM Router** supports ordered fallback chains; note it defaults to retry-*then*-fallback, so it is fail-fast only when retries are set to zero.

5 Lifecycle Management

5.1 LLM as Wizard of Oz

5.1.1 Example. Topic classification of articles is currently performed by pasting the abstract into an LLM and askint it to clasify the topic. This works but is expensive. Once we are satisfied with the topic taxonomy and have accumulated enough labeled data, we will switch to a dedicated topic detection framework.

5.1.2 Forces. LLMs are general-purpose machines that can attempt almost any text-based task. Building dedicated, efficient solutions requires upfront investment and training data that may not yet exist.

5.1.3 Solution. Use the LLM to perform a task in production while building a more efficient replacement. The LLM serves as the "wizard behind the curtain": convincing to users, allowing early beta-testing and feedback, but intended to be temporary.

Bootstrapping variant: In a more subtle variant, the LLM *generates the training data for its own replacement.* For example, Zeeguu uses an LLM to estimate text difficulty levels. These LLM-generated difficulty labels are being accumulated as training data for a classical classifier that will eventually take over the task.

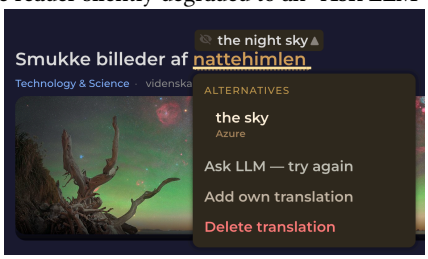
5.1.4 Note. This pattern has a lifecycle relationship with *Escalate to the LLM*: a system may start with the LLM as primary (Wizard of Oz), migrate to a specialized tool as primary, and then keep the LLM as the escalation path, completing a full cycle from LLM-first to LLM-on-demand.

5.1.5 Known Uses.

- **OpenAI Model Distillation** (Stored Completions) captures a large model's production input–output pairs and fine-tunes a smaller model as a drop-in replacement — the pattern shipped as a product feature.
- **Self-Instruct / Alpaca [13]** (Wang et al.; Taori et al., 2023) use a strong LLM to generate the instruction data that fine-tunes a small replacement — the bootstrapping variant.
- **Distilling Step-by-Step [5]** (Hsieh et al., ACL Findings 2023) trains task-specific models up to 700× smaller from LLM-generated rationales.

5.2 Centralized Model Selection

5.2.1 Example. Zeeguu's reader has an on-demand "Ask LLM" translation action. Its model ID, `claude-sonnet-4-20250514`, was hardcoded at three call sites in the translation service and two more in the MWE detector. Anthropic retired that snapshot; every call began returning `404 not_found_error`. The error was swallowed one layer down (the helper returns `None` on any exception), so the endpoint returned a generic `404 "LLM translation failed"` and the reader silently degraded to an "**Ask LLM — try again**" button that could never succeed.



Nothing in the code had changed; a date had passed. The error started showing up.

The fix itself was mechanical: swap to the live snapshot the rest of the codebase already used. But finding it took a dig through production logs, and the swap had to be repeated at five sites.

To avoid this happening again in the future, Zeeguu now keeps every model identifier in one module, keyed by *role* (`WORD_TRANSLATION`, `MWE_DETECTION`, `SIMPLIFICATION`, …) and each resolving to a canonical vendor ID declared once; a feature asks for `models.WORD_TRANSLATION` and never names a snapshot, so the next retirement is a one-line edit.

5.2.2 Forces.

- Model identifiers are volatile in a way ordinary configuration is not
- Providers deprecate and retire dated snapshots on *their* schedule, so a hardcoded ID that works today returns a 404 at some future date, with no change to the code at all.
- Model choice is also cross-cutting: the same handful of IDs get copy-pasted across many features and several cost/quality tiers, so a single retirement breaks many call sites at once, and there is no one place to see, or change, which model each feature uses. Because the failure is a runtime error on a string that looks perfectly valid, it survives code review and type-checking.

5.2.3 Solution. Keep every model identifier in one central module. Declare the canonical vendor IDs once, then expose *role-based aliases* (one per use: translation, classification, simplification…) that resolve to them. Call sites import the role, never the raw string. Surviving a vendor retirement, or moving one feature to a cheaper or faster tier, becomes a one-line change in a file that documents the whole model landscape on one screen.

5.2.4 Notes.

- The mechanism is classical (single source of truth; no magic strings). What makes it an LLM pattern is the *force*: vendor-driven model deprecation turns an innocuous hardcoded string into a scheduled runtime failure. It is the direct defense against the *"old ones regularly deprecated"* property listed among the core LLM forces in the introduction.

- Prefer **role-based** aliases over vendor-based constants. `WORD_TRANSLATION = HAIKU` reads as intent and lets one re-point a single feature's tier without touching others; a bare `HAIKU = "claude-haiku-…"` re-exported everywhere quietly couples unrelated features to the same choice.
- Composes with *LLM Output Provenance*: the identifier a system stamps onto a generated artifact and the identifier it uses to *select* the model at call time should be the same central constant, so the two can never drift apart.
- Composes with *Fail-Fast Provider Chain*: the chain decides the *order* of providers to try; this module names *which model* each provider entry uses.
- *Alternative — AI gateway.* An AI gateway can host the role→model mapping as named aliases in its config, relocating this pattern's registry out of application code. See the broader treatment of what gateways do and do not subsume in *What Makes These Patterns LLM-Specific? → Relationship to LLM Gateways*.

5.2.5 War Story. The same commit that repaired the retirement uncovered a second casualty of the identical disease. Simplified articles were being stamped `ai_model="claude-3-5-sonnet"` while the simplifier had long since moved to Haiku. The provenance field named a model that was no longer even in the pipeline (see *LLM Output Provenance*). Same root cause as the retirement: a model identifier duplicated into a place nobody remembers to update, silently going stale. Centralizing selection lets the provenance stamp and the call-time choice read from the same constant, so they cannot disagree.

5.2.6 Known Uses.

- **LiteLLM model aliases** (`model_alias_map`, proxy `model_group_alias`) map a role/user-facing name to a concrete backend model ID, so swapping models is a one-line central edit.
- **Portkey Model Catalog** references providers and models by a single `@provider/model` alias, changing the backing model in one place.
- The mechanism is the classic *single source of truth*; what makes it LLM-specific is vendor-driven model deprecation (see this pattern's forces).

6 Data Management

6.1 LLM Output Provenance

6.1.1 Example. When the system generates example sentences with a given word to be used in exercises, it stores which model and prompt version produced each result. When a prompt is improved, the system can identify and regenerate stale outputs without reprocessing everything.

6.1.2 Forces. LLM-generated data that enters persistent storage becomes a long-lived asset, but models and prompts improve over time. Without knowing how a piece of data was generated, it cannot be selectively regenerated when better models or prompts become available. Prompts evolve more frequently than model versions and can have a larger impact on output quality.

6.1.3 Solution. Store the full provenance tuple alongside every LLM-generated artifact: (model version, prompt version, generated output, timestamp). This enables selective regeneration (e.g., *"re-run everything produced by prompt v2 with the improved prompt v3"*) and quality auditing.

6.1.4 Notes.

- The key insight is that the prompt is at least as important to version as the model: a prompt change can completely alter output format, quality, or behavior even with the same model.

- This is also critical for the Wizard of Oz pattern: when accumulating LLM-generated labels as training data for a classical replacement, provenance tracking lets one exclude data produced by a prompt version that was later found to be noisy or biased.

- The identifier stamped for provenance should be the *same* constant the code uses to select the model, kept in one place (*Centralized Model Selection*). When the two are separate literals, the selection can move to a new model while the provenance field keeps naming the old one. A field that names a model no longer in the pipeline is worse than no field at all.
- Implicit provenance: Keep model names and prompt versions as constants in code. When one needs to know what generated a piece of data, correlate its `created_at` timestamp with git history to determine which model/prompt was deployed at that time. However, this works for simpler systems where there is a single model/prompt active at any time. A system using alternative prompts, e.g. for A/B testing, will have to track provenance explicitly. Also, explicit tracking makes data analysis faster, and ensures that data is self-describable.

6.1.5 Known Uses.

- **MLflow Prompt Registry** stores commit-versioned prompts with their model configuration and tracks prompt-version ↔ app-version ↔ trace lineage for reproducibility and selective regeneration.
- **LangSmith** creates an immutable commit hash per prompt version and records which prompt/model produced each traced output.
- Both confirm this pattern's core claim that the *prompt* deserves versioning as much as the model.

6.2 Soft Invalidation of LLM Artifacts

6.2.1 Example. When the prompt that generates audio lesson scripts was improved, the ~900 stored `audio_lesson_meaning` rows produced under the previous prompt were neither regenerated eagerly nor deleted. Instead, each affected row received a `deprecated_at` timestamp, and the cache-lookup helper (`AudioLessonMeaning.find()`) was gated to skip deprecated rows. New daily lessons request a fresh row and trigger regeneration under the new prompt; existing daily lessons that already reference a deprecated row keep playing their old audio without breaking.

6.2.2 Forces. When a prompt or model improves, the obvious responses each have a serious drawback: - *Regenerate everything eagerly*: expensive, floods generation queues if affected rows number in the thousands, and pays for content that may never be re-requested. - *Delete the stale rows*: breaks any downstream object that references them by id (history, analytics, user-visible past sessions). - *Leave the stale rows in place and accept future reuse*: silently propagates the old, known-suboptimal quality.

None of these are good defaults for production systems where LLM-generated artifacts are referenced from user-visible history and are also targets for reuse.

6.2.3 Solution. Mark stale rows as deprecated rather than mutating or removing them. Gate the cache-lookup / reuse path to skip deprecated rows, forcing fresh generation on next demand. Existing references to a deprecated row remain

valid (the row keeps its content for historical playback), but no new consumer picks it up. Regeneration cost is paid lazily, amortized over normal access patterns, and only for content that is actually requested again.

6.2.4 Notes.

- This pattern is forward-only: it gates *reuse*, not *playback*. A user replaying an old lesson hears the old (lower-quality) version. That is usually preferable to a silent content swap mid-history.
- Works best when content has a clear "next access triggers regeneration" entry point. If consumers cache aggressively further downstream, the deprecation flag has to propagate to those layers too.
- Composes naturally with *LLM Output Provenance*: provenance answers "which rows are stale?", soft invalidation answers "what do I do with them once I know?".
- **Prerequisite: artifact identity must follow the row, not the upstream key.** If the on-disk artifact (audio file, image, embedding) is named after the *source identity* (e.g. `meaning_id`) rather than the *row identity* (e.g. `audio_lesson_meaning.id`), the regenerated row's artifact overwrites the deprecated row's artifact on the same path, defeating the historical-playback guarantee. Zeeguu encountered this concretely: meaning-lesson audio files were keyed by `meaning_id`, so regeneration silently replaced the audio referenced by old daily lessons. A separate change re-keyed those files by row id to make Soft Invalidation safe. This small structural requirement may deserve being a pattern in its own right (working title: *artifact identity = row identity*).

6.2.5 Known Uses.

- **`stale-while-revalidate`** (IETF RFC 5861) is the same mechanic in HTTP caching: keep a stale entry usable and revalidate it lazily on next demand rather than eagerly regenerating.
- The **soft-delete / tombstone** database idiom marks a row with a timestamp and gates every read (`WHERE deleted_at IS NULL`), so the row stays intact for existing references but drops out of the active path — structurally identical to a `deprecated_at` gate.
- *Analogues, not exact instances.* Both capture the mechanism, but we did not find a documented LLM system that combines mark-deprecated + gate-reuse + **keep-old-rows-resolvable-for-user-history** + lazy regeneration; the history-preservation facet appears novel, which is why this stays a candidate.

7 Quality Assurance

7.1 LLM-Checking-LLM

7.1.1 Example. After generating contextual example sentences for vocabulary words, a second LLM call reviews the examples for accuracy, naturalness, and appropriate difficulty level.

7.1.2 Forces. LLMs are imprecise generators, but verification of specific properties (e.g., grammatical correctness) is a more constrained task than open-ended generation (e.g., text simplification). A second, focused LLM call can catch errors that the first, more complex call introduced.

7.1.3 Solution. Use one LLM call to generate a result, then use a separate LLM call to check or refine it. The verification prompt can be simpler and more focused than the generation prompt.

7.1.4 Note. This is distinct from ensemble methods or chain-of-thought: the key insight is that checking is a fundamentally easier task than generating, and this asymmetry can be exploited architecturally.

7.1.5 Known Uses.

- **LLM-as-a-judge [14]** (Zheng et al., NeurIPS 2023) uses a separate strong LLM to score another model's open-ended outputs.
- **Self-Refine [8]** (Madaan et al., NeurIPS 2023): a separate pass critiques and refines the first output.
- **G-Eval [7]** (Liu et al., 2023), shipped in eval frameworks such as **DeepEval**, uses a separate chain-of-thought judge call to grade generated output.
- *Contrast.* Unlike self-critique on the same reasoning, this pattern uses a differently-prompted call to check a *different property* (see *Related Work* on Stechly et al.).

7.2 LLM Content Validation Tracking

7.2.1 Example. A word-translation pair can exist at several trust levels: generated by Google Translate (unverified), verified by an LLM checker (auto-verified), implicitly accepted by a user who practiced it without complaint, or explicitly corrected by a user. Each level carries different confidence, and the system can make different decisions depending on the validation state: for example, only including explicitly confirmed translations in exercises, while using auto-verified ones for reading assistance where the stakes are lower.

7.2.2 Forces. Once LLM-generated data is persisted, it becomes indistinguishable from human-verified data unless explicitly marked. Downstream features and users may silently treat unverified AI output as ground truth. Over time, the system accumulates a mix of trusted and untrusted data with no way to tell them apart.

7.2.3 Solution. Maintain an explicit, queryable validation state for all LLM-generated content in the data model. Never let LLM-generated content silently become trusted data. The validation state may be a simple flag, but more often it is a spectrum or state machine reflecting different levels of trust (e.g., unverified → auto-verified → implicitly accepted → explicitly confirmed by user; alternatively, one could track the number of users that have validated a given content).

7.2.4 Notes.

- This pattern complements *LLM Output Provenance*. Together they answer two essential questions about any piece of LLM-generated data: how was it produced? and has anyone confirmed it's correct? The right granularity of validation tracking depends on the domain: some systems may need binary (verified/not), others may need multiple validators with agreement thresholds, and others, like Zeeguu, benefit from a trust spectrum that reflects different forms of implicit and explicit user feedback.

- Implicit validation: One could argue that "if a user practiced a word without complaint, it's implicitly validated"

7.2.5 Known Uses.

- **Argilla** records carry an explicit status lifecycle (pending → draft → submitted, plus validated/discarded), so model suggestions are not trusted until a human validates.
- **Label Studio** tracks a per-task ground-truth flag and review state, distinguishing verified gold data from unreviewed predictions.
- **Snorkel [10]** (Ratner et al., VLDB 2017) attaches probabilistic confidence to programmatically generated labels rather than treating them as ground truth — a confidence spectrum rather than a discrete state machine.

- *Note.* These are data-labeling platforms; a documented, in-product *LLM-output* trust-state lifecycle (as opposed to human-annotation state) remains thinly evidenced — one reason we keep this among the less-settled patterns.

7.3 Hybrid Classical+LLM Pipeline

7.3.1 Example. Multi-word expression (MWE) detection runs Stanza's dependency parser first, as a cheap high-recall gate. If Stanza flags no candidate in a sentence, the LLM is never called. When it does flag one, an LLM re-analyzes the whole sentence and its verdict is used (and if the LLM finds no expression, its precision is trusted over Stanza's). The LLM therefore runs on only the fraction of sentences that might contain an expression, rather than on every sentence.

7.3.2 Forces. Classical NLP tools (dependency parsers, POS taggers, rule-based extractors) are fast and deterministic but miss edge cases. LLMs handle edge cases well but are expensive to run on every input. Neither alone achieves both high recall and high precision.

7.3.3 Solution. Run the cheap classical tool first, as a high-recall gate: invoke the LLM only when the classical stage fires, and skip it otherwise (the common case, and the main cost saving). When it fires, let the LLM make the precision decision. The two are not alternatives: the classical stage controls *when* the LLM runs; the LLM controls *what counts*.

7.3.4 Tradeoff. Requires maintaining two systems, but the cost savings from not sending every input to the LLM typically justify the complexity.

7.3.5 Note. Close kin to *Escalate to the LLM* and to a model cascade: all three run a cheap step first and call the LLM selectively. The difference is the trigger. Escalate uses the cheap tool's answer and reaches for the LLM only when it is inadequate (a failure, or user dissatisfaction); here the cheap tool gates on detected difficulty (a flagged candidate) and the LLM's verdict replaces it, as in a confidence-based cascade.

7.3.6 Known Uses.

- **RankGPT [12]** (Sun et al., EMNLP 2023) uses BM25 to retrieve ~100 candidates, then an LLM for listwise reranking — classical high-recall generator + LLM precision filter.
- **spaCy-llm** (Explosion) is designed to mix LLM components with classical/rule-based ones in a single pipeline.
- **Retrieve-then-rerank** (e.g. Cohere Rerank) keeps a high-recall first-stage search and adds a neural precision reranker.

7.4 Defensive Output Parsing

7.4.1 Example. Multi-word-expression detection asks the LLM for a JSON array but refuses to blindly trust that it will get one.

- `_parse_response` first strips any markdown code fence (the model often wraps the JSON in one), tries `json.loads`, and on failure regex-extracts the last JSON array in the text (models tend to add a preamble), parses that, and checks for a bare `[]`.
- If nothing parses, it logs the raw text and returns `[]` rather than raising.
- A separate `_validate_groups` step then checks the parsed shape (token indices in range) before anything downstream uses it.

The same layered try / extract / validate / fall-back appears in the translation validator, the simplification service, and the example generators.

7.4.2 Forces.

- A fixed output format is required (JSON, a delimited record)
- Format compliance is probabilistic
- A naive parse on the critical path can turn a formatting slip into a failed request

7.4.3 Solution. Do not trust the raw output's shape.

Parse in layers: strip known wrappers, attempt a lenient parse, extract the expected structure from the surrounding text if that fails, validate the parsed shape (types, ranges, required fields), and then degrade gracefully (a default, a skip, a retry, or the next provider) instead of raising.

Keep the strictness in code, where it is deterministic and testable, rather than in ever-longer prompt instructions.

7.4.4 Notes.

- A slip degrades a single result instead of failing the request. It composes with a one-shot retry and with *Fail-Fast Provider Chain* (a parse failure can trigger the next provider).
- Provider "JSON mode" or function-calling reduces malformed output but does not remove the need to validate the shape before use.
- Related to *Deterministic Postprocessing*, which repairs a specific, known formatting defect; this pattern is the broader stance of not trusting the structure at all.

7.4.5 Known Uses.

- **Instructor** validates responses against a Pydantic schema and auto-retries on validation failure, feeding the error back for correction.
- **LangChain** `OutputFixingParser` / `RetryOutputParser` re-invoke the model to repair malformed structure on a parse failure.
- **OpenAI Structured Outputs** guarantees schema-conformant JSON yet still requires handling truncation (`finish_reason`) and `refusal` — illustrating this pattern's point that JSON mode reduces but does not remove validation.

7.5 Deterministic Postprocessing

7.5.1 Example. LLM-simplified article summaries consistently ended with a Unicode ellipsis (…), making every home-card preview read as an unfinished sentence. One option was to add a "do not end with ellipsis" instruction to the simplification prompt; the chosen option was a five-line regex stripping any trailing … or `. . .` + at serialization time. The regex handles every case at 100%, including the ~60k pre-existing rows in the database that no prompt change could retroactively touch.

7.5.2 Forces. When LLM output has a deterministic formatting defect (a stable trailing string, a known preamble, leaked markdown in a plain-text field, trailing whitespace), the obvious instinct is to fix it in the prompt. But: - Prompt compliance is probabilistic; the same constraint in code is 100%. - Prompt tokens cost money on every call and can distract the model from the actual semantic task. - Prompt changes do not affect rows already in the database. - Code is testable and reviewable; prompt instructions are not.

7.5.3 Solution. Enforce deterministic constraints in code, at the post-processing or serialization boundary. Reserve prompt instructions for things that genuinely require model judgment.

7.5.4 Notes. The boundary between "deterministic" and "semantic" is the test. *Strip a trailing …*: deterministic, do it in code. *Don't mention the user's name*: semantic, the model has to enforce. When the deterministic rule list grows long, that is itself a signal that the task is poorly scoped, not that the prompt needs more rules.

7.5.5 Known Uses.

- *Adjacent structural cousin.* Structured-output libraries such as **Outlines** constrain generation so the format is valid upfront, and explicitly contrast this with the common practice of "fixing bad outputs after generation using parsing, regex, or fragile code" — evidence that code-side repair is widespread, though Outlines *prevents* rather than *repairs*.
- *Largely best-practice / folklore.* We found no source that names this specific rule — repair a *deterministic* defect in code, not the prompt, because code is 100% reliable and also fixes already-stored rows. The literature documents the stronger cousin (constrained decoding / structured outputs) and generic output sanitization, so we present this as its own contribution and cite these as adjacent, not prior.

8 Candidate Patterns

8.1 Temperature as Task Selector

8.1.1 Example. Translation validation uses temperature 0 for deterministic yes/no judgments. Audio lesson script generation uses temperature 0.8 to produce varied, natural-sounding dialogues. The same model serves both purposes with different configuration.

8.1.2 Forces. LLMs exhibit different behaviors at different temperature settings. Classification and validation tasks benefit from deterministic outputs (low temperature), while creative generation benefits from variety (higher temperature). Using a single temperature for all tasks either sacrifices reliability or creativity.

8.1.3 Solution. Systematically vary temperature based on task type. Use temperature 0–0.3 for tasks requiring consistency (validation, classification, structured extraction). Use temperature 0.7–1.0 for tasks requiring creativity (dialogue generation, example variety).

8.1.4 Note. This pattern acknowledges that a single LLM can behave as multiple "virtual components" depending on configuration: deterministic validator vs. creative generator.

8.1.5 Known Uses.

- **Vendor guidance**: Anthropic's Messages API advises temperature near 0 for analytical/multiple-choice tasks and near 1 for creative ones; the OpenAI Cookbook hard-codes `temperature=0` for classification.
- **Azure OpenAI** guidance recommends 0–0.3 for extraction/categorization and 0.7–1.0 for creative generation.
- *Caveat worth keeping.* Renze & Guven [11] (EMNLP Findings 2024) found temperature 0.0–1.0 has *no* statistically significant effect on problem-solving accuracy — evidence that this pattern's justification is determinism/consistency and output variety, not correctness.

8.2 Prompt Injection Containment

8.2.1 Example. Two Zeeguu surfaces feed *untrusted text* into a prompt, and they contain it with very different strength.

Audio-lesson suggestions (strong containment). A user types a free-text topic or situation for a listening lesson, which then drives an expensive multi-step generation pipeline (script → text-to-speech → stored artifact). `suggestion_validator` contains the input in layers: hard length caps (2–80 characters) reject oversized payloads before any model runs; then a cheap, temperature-0 LLM *gatekeeper* classifies the input (`invalid` / `niche` / `general`) **and canonicalizes it to a short 2–5 word normalized topic** in the user's native language. The downstream generator consumes the *canonical form*, not the raw text — so an injected instruction ("ignore your rules and…") is either rejected as `invalid` or collapsed into a 2–5 word topic, and never reaches the generator verbatim.

On-demand "Ask LLM" translation (light containment). The learner's clicked word and the surrounding article sentence are inserted into the prompt. The context here is third-party web content the app ingested, so it is *also* untrusted. Containment is implicit rather than explicit: the task is deliberately narrow ("translate the word in context, 1–3 words"), the expected output is tightly constrained, and *Defensive Output Parsing* rejects anything that is not a short translation. This bounds the blast radius rather than sanitizing the input.

8.2.2 Forces.

- An LLM cannot reliably separate *data* from *instructions*: any untrusted text in the prompt — typed by a user, or lifted from third-party content the app ingests — can carry instructions the model may follow.
- In a *component* (not a chatbot), the output feeds downstream automated steps that generate, synthesize, and *persist* artifacts. A successful injection is therefore not merely an off-topic reply; it can produce unintended stored content, waste expensive compute, or emit unsafe output under the application's name.
- Legitimate and injected text share one channel — the prompt — so you cannot filter injection out by transport; you must *reduce, constrain, or scope* the untrusted text instead.
- Perfect sanitization is impossible. Defenses are layered and probabilistic, and each extra gatekeeping call has a cost that must be traded against the consequence it guards.

8.2.3 Solution. Never let raw untrusted text flow into a consequential prompt as-is. Contain it with cheap, layered defenses, strongest where the blast radius is largest:

- **Structural caps first** — length / character limits reject obvious payloads before any model runs (free, deterministic).
- **Gate-and-canonicalize** — a cheap, low-temperature classifier (LLM or classical) validates the input *and replaces it with a short normalized canonical form*; downstream prompts consume the canonical form, not the original. Reducing untrusted free text to a 2–5 word token is itself a strong injection defense: there is almost no room left to smuggle instructions.
- **Narrow the task and constrain the output** — a prompt that can only emit a 1–3 word translation, validated by *Defensive Output Parsing*, bounds what a successful injection achieves even without sanitizing the input.
- **Fail safe** — on validator error, degrade to the most restrictive behavior the feature can tolerate, never to "pass the raw text through."

Match containment strength to consequence: the audio pipeline (multi-step, persists artifacts, real cost) earns a dedicated gatekeeper call; a throwaway inline translation leans on scoping and output validation.

8.2.4 Notes.

- Composes with *Defensive Output Parsing* — constraining and validating the *output* is the second half of containment: limit what goes in *and* what can come out. It also composes with *LLM-Checking-LLM*: the gatekeeper is a checker specialized for safety rather than quality.
- **Honest gap:** the article-context vector is only *partially* contained. Article text is third-party and could carry injection; today's defense there is task-narrowing plus output constraint, not context sanitization. We flag this rather than claim full coverage.
- **Fail-open vs. fail-closed:** `suggestion_validator` deliberately *fails open* — on LLM error it lets the suggestion through with only basic sanitization, trading safety for availability on a low-stakes feature. A higher-stakes surface should fail *closed*. Making that choice explicit per feature is part of the pattern.

8.2.5 Relationship to LLM Gateways. Some gateways offer prompt-injection / content guardrails (Portkey guardrails, LiteLLM hooks, dedicated screens such as Lakera or prompt-shield features) that can run a screening pass at the gateway. But the strongest containment here is not a generic screen — it is *domain canonicalization* (reduce the input to a 2–5 word topic in the user's language) and *task-narrowing*, both of which require the application's knowledge of what a legitimate input looks like. A gateway can add a screening layer; it cannot know that "a valid audio-lesson topic is a 2–5 word phrase" or "a valid translation is 1–3 words." As elsewhere (see *What Makes These Patterns LLM-Specific? → Relationship to LLM Gateways*): gateway = generic guardrail; pattern = domain-specific reduction and scoping.

8.2.6 Status. Partly implemented: the audio-lesson suggestion validator is a full instance; translation relies on task-narrowing plus defensive parsing; the third-party article-context vector is only partially addressed. Included as a candidate to discuss the boundary with *Defensive Output Parsing* and *LLM-Checking-LLM* — whether "contain untrusted input" and "validate untrusted output" are two halves of one pattern or two.

8.2.7 Known Uses.

- **OWASP Top 10 for LLM Applications** (LLM01: Prompt Injection) codifies the threat and lists mitigations matching this pattern's layers: constrain behavior, validate/constrain output, filter input/output, and segregate untrusted content from trusted prompts.
- **Simon Willison's Dual-LLM pattern** handles untrusted text only in a tool-less quarantined LLM while a controller substitutes variable tokens, so raw untrusted content never reaches the privileged LLM — a stronger sibling of our canonicalize-before-use step.
- **NVIDIA NeMo Guardrails** "input rails" screen user messages for jailbreak/injection before they reach the LLM.

9 What Makes These Patterns LLM-Specific?

Some of these patterns echo general distributed systems wisdom (batching, fallback, redundant dispatch). What makes them distinctly relevant to LLM integration is the specific combination of forces:

- **Cost structure**: per-token pricing with high fixed prompt overhead, unlike flat-rate API calls
- **Non-determinism**: same input can yield different outputs, necessitating verification chains

- **Asymmetry between generation and verification**: checking is easier than producing
- **General-purpose capability**: the same component can serve as prototype, primary, or fallback
- **Quality–cost–latency tradeoff space**: uniquely wide compared to traditional APIs

9.1 Relationship to LLM Gateways

Several of these patterns (*Fail-Fast Provider Chain*, *Centralized Model Selection*, and hot-path caching) are being commoditized into LLM-gateway infrastructure (LiteLLM, Portkey, Cloudflare AI Gateway). This does not invalidate them as patterns; it relocates their *implementation* from application code into shared infrastructure. A pattern documents a recurring solution whether it is hand-rolled or shipped by a gateway, and knowing the pattern is precisely what lets one judge whether a given gateway implements it well (e.g. most gateways do *not* provide *Multiplexed Dispatch* with alternative-caching). Connection pooling remained a pattern long after every ORM started shipping one.

The recurring shape across all of these is that the gateway supplies a *mechanism* while the pattern owns the *intent, the domain join, and the decision the mechanism drives*. This is sharpest in *Per-User Consumption Budget*, where the gateway can throttle and cap but only the application knows which resource to bound and what the user should get when the budget is exhausted.

10 Related Work

To our knowledge, no peer-reviewed work presents a catalog of architectural patterns for integrating LLMs as components into existing production systems, grounded in real deployment experience and described using the standard pattern format (context, forces, solution, consequences). Our patterns addressing lifecycle management (Wizard of Oz), cost optimization (Pre-computation, Prompt Amortization, Escalate to the LLM, Multiplexed Dispatch), quality assurance (LLM-Checking-LLM, Validation Tracking), and data management (Provenance) appear to be novel contributions.

Several practitioner-oriented resources discuss patterns for building LLM-based systems, but they operate at a different level of abstraction and lack the grounding in a specific production system that we aim to provide.

10.1 Practitioner resources

Eugene Yan's "**Patterns for Building LLM-based Systems & Products**" (2023, blog post) identifies seven patterns: Evals, RAG, Fine-tuning, Caching, Guardrails, Defensive UX, and Collecting User Feedback. These address the question "how do I build an LLM product?" They describe the overall stack for LLM-native applications. Our patterns address a different question: "I have an existing system, and I want to add LLM capabilities as a component: how do I manage cost, quality, latency, and lifecycle?" There is some overlap on caching/pre-computation, but our treatment focuses on prompt amortization and user-need anticipation rather than semantic similarity caching. Yan's work is not peer-reviewed.

ThoughtWorks' "**Emerging Patterns in Building GenAI Products**" (Fowler et al., martinfowler.com) and "Engineering Practices for LLM Applications" focus on operational concerns: testing, evaluation, guardrails, and RAG pipelines. These are primarily about LLMOps rather than the architectural decisions for embedding LLMs as components within an existing application.

Andreessen Horowitz's "**Emerging Architectures for LLM Applications**" (2023) provides a reference architecture for the LLM infrastructure stack (embedding pipelines, vector databases, orchestration layers, agents). Again, this targets LLM-native products rather than LLM integration into existing systems.

Books. "**LLM Design Patterns**" (Huang, Packt, 2024) and "LLMs in Enterprise" (Menshawy & Fahmy, Packt, 2025) cover model-level patterns (fine-tuning, quantization, inference optimization, RAG) and enterprise deployment concerns. They do not address application-level integration patterns such as the lifecycle management (Wizard of Oz → specialized tool → Escalate to the LLM), prompt amortization, or LLM output provenance that we identify.

10.2 Academic surveys.

There is a large body of work on **using LLMs *for* software engineering tasks**: code generation, bug repair, testing, requirements engineering (see surveys by Fan et al., 2023; Zhang et al., 2024). However, these focus on LLMs as tools for developers, not on the engineering challenges of integrating LLMs as runtime components within production software.

A recent **systematic literature review on software architecture and LLMs** (Schmid et al., 2025) found only 18 relevant papers and noted that LLM-based software design remains an open research direction. None of the surveyed academic work addresses the specific architectural patterns for managing cost, quality, latency, and lifecycle when LLMs serve as components in existing applications.

LLM self-verification. For the *LLM-Checking-LLM* pattern, there is relevant work on **LLM self-verification**. Gero et al. (2023) demonstrated that self-verification improves clinical information extraction accuracy, explicitly building on the asymmetry between verification and generation. However, Stechly et al. (2024) showed that self-critique fails for formal reasoning tasks, finding significant performance collapse with self-verification but gains with external verification. Our pattern differs from both in that it uses separate, differently-prompted LLM calls for generation and verification of different properties (e.g., text simplification followed by grammatical correction), rather than asking an LLM to verify its own reasoning.

11 Limitations and Future Work

The patterns in this catalogue have been extracted from a single production system, Zeeguu, in the language-learning domain. We have argued throughout that the underlying forces — per-token cost, multi-second latency, non-determinism, a rapidly shifting provider landscape, and general-purpose capability — are not specific to language learning, and that the patterns should therefore generalise to other interactive applications that surface LLM-generated text to end users. Whether this is borne out in practice is an empirical question we cannot fully answer from a single case study. The most direct way to validate or refute the catalogue is to gather complementary examples — and counter-examples — from practitioners working in other domains. A PLoP writers' workshop is one venue for exactly this kind of community refinement; mining open-source repositories for additional instances of these patterns (and patterns we missed) is a natural follow-up.

A second limitation is that the catalogue reports patterns we have found useful but does not yet quantify their impact systematically. Anecdotal cost and latency improvements are reported per pattern; a more rigorous deployment-level evaluation — measuring, for example, how much of Zeeguu's LLM bill is attributable to which patterns, or how much user-perceived latency *Multiplexed Dispatch* removes — is future work.

Finally, we expect the catalogue to be incomplete. Several proto-patterns are listed in *Candidate Patterns*; others will likely surface only through engagement with practitioners working at different scales, in different domains, and with different LLM providers.

12 Conclusion

This paper has presented a catalogue of architectural patterns for integrating LLMs as components into existing user-facing applications — a setting that has received much less attention than the design of LLM-native products, despite arguably affecting a larger share of working software engineers. The patterns address recurring concerns around cost, latency, lifecycle, data management, and quality assurance, and were drawn from real production experience on the Zeeguu platform. They are intended as a starting point for community refinement rather than a closed taxonomy: practitioners working on LLM integration in other domains are warmly invited to validate, refute, extend, or replace individual patterns, and to contribute new ones the catalogue does not yet contain.

13 Acknowledgments

Thanks to Cesare Pautasso for the discussion that prompted this pattern.

References

[1] Fu Bang. 2023. GPTCache: An Open-Source Semantic Cache for LLM Applications Enabling Faster Answers and Cost Savings. In *Proceedings of the 3rd Workshop for Natural Language Processing Open Source Software (NLP-OSS) at EMNLP*. https://aclanthology.org/2023.nlposs-1.24/

[2] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *Transactions on Machine Learning Research (TMLR)* (2023). https://arxiv.org/abs/2305.05176

[3] Zhoujun Cheng, Jungo Kasai, and Tao Yu. 2023. Batch Prompting: Efficient Inference with Large Language Model APIs. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: Industry Track (EMNLP)*. https://arxiv.org/abs/2301.08721

[4] Jeffrey Dean and Luiz André Barroso. 2013. The Tail at Scale. *Commun. ACM* 56, 2 (2013), 74–80. https://cacm.acm.org/research/the-tail-at-scale/

[5] Cheng-Yu Hsieh, Chun-Liang Li, Chih-Kuan Yeh, Hootan Nakhost, Yasuhisa Fujii, Alexander Ratner, Ranjay Krishna, Chen-Yu Lee, and Tomas Pfister. 2023. Distilling Step-by-Step! Outperforming Larger Language Models with Less Training Data and Smaller Model Sizes. In *Findings of the Association for Computational Linguistics (ACL)*. https://arxiv.org/abs/2305.02301

[6] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*. https://arxiv.org/abs/2004.12832

[7] Yang Liu, Dan Iter, Yichong Xu, Shuohang Wang, Ruochen Xu, and Chenguang Zhu. 2023. G-Eval: NLG Evaluation using GPT-4 with Better Human Alignment. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2303.16634

[8] Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. 2023. Self-Refine: Iterative Refinement with Self-Feedback. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2303.17651

[9] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. 2024. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665* (2024). https://arxiv.org/abs/2406.18665

[10] Alexander Ratner, Stephen H. Bach, Henry Ehrenberg, Jason Fries, Sen Wu, and Christopher Ré. 2017. Snorkel: Rapid Training Data Creation with Weak Supervision. *Proceedings of the VLDB Endowment* 11, 3 (2017). https://arxiv.org/abs/1711.10160

[11] Matthew Renze and Erhan Guven. 2024. The Effect of Sampling Temperature on Problem Solving in Large Language Models. In *Findings of the Association for Computational Linguistics: EMNLP*. https://arxiv.org/abs/2402.05201

[12] Weiwei Sun, Lingyong Yan, Xinyu Ma, Shuaiqiang Wang, Pengjie Ren, Zhaochun Chen, Dawei Yin, and Zhaochun Ren. 2023. Is ChatGPT Good at Search? Investigating Large Language Models as Re-Ranking Agents. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. https://arxiv.org/abs/2304.09542

[13] Rohan Taori, Ishaan Gulrajani, Tianyi Zhang, Yann Dubois, Xuechen Li, Carlos Guestrin, Percy Liang, and Tatsunori B. Hashimoto. 2023. Stanford Alpaca: An Instruction-following LLaMA Model. https://github.com/tatsu-lab/stanford_alpaca. https://github.com/tatsu-lab/stanford_alpaca

[14] Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*. https://arxiv.org/abs/2306.05685